

CS100 Computer Programming
Spring 2023
Midterm Exam

Instructors: Lan Xu, Yuexin Ma

Time: May 15th 8:15-10:15

INSTRUCTIONS

Please read and follow the following instructions:

- You have 120 minutes to answer the questions.
- You are not allowed to bring any books or electronic devices including regular calculators.
- You are not allowed to discuss or share anything with others during the exam.
- You may bring two pieces of A4-sized double-sided cheat sheets.
- You should write the answer to every problem in the dedicated box **clearly**.
- You should write **your name and your student ID** as indicated on the top of **each page** of the exam sheet.

Name	
Student ID	

Please write your answers to the multiple choices questions in the following table.

(1)	(2)	(3)	(4)	(5)
CD	ABCE	BC	BC	C
(6)	(7)	(8)	(9)	(10)
AE	B	AB	AD	ACDE
(11)	(12)	(13)	(14)	(15)
BDF	AD	AB	AB(C)	ACD
(16)	(17)	(18)	(19)	(20)
C	AD	B	BC	B

1. (60 points) Multiple Choices

Each of the following questions has **one or more** correct choices.

You will get half of a question's points if you choose a non-empty proper subset of its correct choices.

The questions marked "[C]" are based on C17. The questions marked "[C++]" are based on C++17.

(1) (3') [C] Read the following code.

```
#include <stdio.h>
int x = 42;           // (1)
int incre(int x) {    // (2)
    return x + 1;
}
int main(void) {
    int x;             // (3)
    x = 15;
    if (x == 15) {
        int x = incre(10);
        printf("%d\n", x++); // (4)
    }
    printf("%d\n", incre(x)); // (5)
    return 0;
}
```

- A. The x in (1) and (2) refer to the same variable.
 - B. In (3), the value of x is 42 because it was initialized to 42 in (1).
 - C. The value printed by (4) is 11.
 - D. The value printed by (5) is 16.
 - E. The value printed by (5) is 13.
- (2) (3') [C] Which of the following statements is/are true?
- A. A string is an array (using either static memory or dynamic memory) of **chars** with a null character '\0' at the end.
 - B. To obtain the length of a string, use **strlen(s)**.
 - C. **char *cp1 = "hello";**
char cp2[] = "hello";
cp1 is a pointer that points to the string literal "hello". **cp2** is an array whose elements are copied from the string literal "hello".
 - D. With **cp1** and **cp2** defined as above, both **cp1[0] = 'b';** and **cp2[0] = 'b';** correctly modify the first character of that string.
 - E. With **cp2** defined as above, **sizeof(cp2)** is equal to 6.
- (3) (3') [C] Suppose we have the following **struct** that stores the basic information of a student.

```
struct student {
    bool gender; // use true/false to represent male/female
    char *name;
    int id;
};
```

We want to define a function **student_new** that creates a **student** in dynamic memory and returns a pointer to it, and a function **student_delete** that destroys a **student** and deallocates

its memory. It is unspecified whether `student_new` should copy the student's name, but the behaviors of `student_new` and `student_delete` should match with each other. Select the correct definitions that do not lead to compile-errors, memory leaks or undefined behaviors.

- A. `struct student *student_new(bool is_male, char *name, int i) {`
 `struct student *student`
 `= malloc(sizeof(bool) + sizeof(char *) + sizeof(int));`
 `student->gender = is_male;`
 `student->name = name; // The name is not copied.`
 `student->id = i;`
 `return student;`
`}`
 `void student_delete(struct student *student) {`
 `free(student);`
 `}`
- B. `struct student *student_new(bool is_male, char *name, int i) {`
 `struct student *student = malloc(sizeof(*student));`
 `*student = (struct student) {`
 `.gender = is_male,`
 `.name = name, // The name is not copied.`
 `.id = i`
 `};`
 `return student;`
`}`
 `void student_delete(struct student *student) {`
 `free(student);`
 `}`
- C. `struct student *student_new(bool is_male, char *name, int i) {`
 `struct student *student = malloc(sizeof(struct student));`
 `student->gender = is_male;`
 `student->name = malloc(strlen(name) + 1);`
 `strcpy(student->name, name); // The name is copied.`
 `student->id = i;`
 `return student;`
`}`
 `void student_delete(struct student *student) {`
 `free(student->name);`
 `free(student);`
 `}`
- D. `struct student *student_new(bool is_male, char *name, int i) {`
 `struct student *student`
 `= malloc(sizeof(bool) + sizeof(int) + strlen(name) + 1);`
 `student->gender = is_male;`
 `strcpy(student->name, name);`
 `student->id = i;`
 `return student;`
`}`
 `void student_delete(struct student *student) {`

```

    free(student->name);
    free(student);
}

```

- (4) (3') [C] Suppose the following code appears very frequently in our program.

*// ptr is a const char *, pointing to some position in a string.*

```

int depth = 0;
while (*ptr != '\0' && !some_condition(*ptr, depth))
    ++ptr;

```

Now we want to write a function to do this job, in order to make our code more readable and less repetitive. Select the correct designs. (Both the function definition and the usage should be correct.)

A.

```
void move_ptr(const char *ptr) {
    int depth = 0;
    while (*ptr != '\0' && !some_condition(*ptr, depth))
        ++ptr;
}
```

To use this function: `move_ptr(ptr);`

B.

```
void move_ptr(const char **ptr) {
    int depth = 0;
    while (**ptr != '\0' && !some_condition(**ptr, depth))
        ++*ptr;
}
```

To use this function: `move_ptr(&ptr);`

C.

```
const char *move_ptr(const char *ptr) {
    int depth = 0;
    while (*ptr != '\0' && !some_condition(*ptr, depth))
        ++ptr;
    return ptr;
}
```

To use this function: `ptr = move_ptr(ptr);`

D.

```
void move_ptr(const char **ptr) {
    int depth = 0;
    while (**ptr != '\0' && !some_condition(**ptr, depth))
        ++*ptr;
}
```

To use this function: `move_ptr(*ptr);`

- (5) (3') [C] Which of the following statements is/are true?

A. `printf(NULL)` has the same effect as `printf("%p", NULL)`, which prints something indicating a null pointer value (possibly "null").

B. Suppose `f` is a `float`. `printf("%d", f)` has the same effect as `printf("%d", (int)f)`. For example, `printf("%d", f)` prints "3" if the value of `f` is 3.14.

C.

```
int *ptr = NULL;
free(ptr);
```

This code does not lead to undefined behaviors.

D.

```
int ival = 100000000;
long long llval = ival * ival;
```

Suppose `sizeof(int) == 4` and `sizeof(long long) == 8`. This code does not have undefined behaviors.

Solution:

- A `printf` expects the first argument to be a string. Passing `NULL` to it is undefined behavior.
- B If the conversion specifier does not match the argument type, the behavior is undefined.
- C `free(NULL)` does not do anything.
- D Overflow happens when calculating `ival * ival`, no matter what type of variable is the result stored into.

(6) (3') [C] Read the following code.

```
void mystery(char *str, int n) {
    if (n <= 1) {
        return;
    } else if (n == 2) {
        char tmp = str[0];
        str[0] = str[1];
        str[1] = tmp;
        return;
    } else {
        mystery(str, n / 2);
        mystery(str + n / 2, n - n / 2);
    }
}
```

```
char str[] = /* SOME VALUE */;
mystery(str, strlen(str));
printf("%s\n", str);
```

Which of the following statements is/are true?

- A. If `/* SOME VALUE */` is `"01234567"`, the code will print `10325476`.
- B. If `/* SOME VALUE */` is `"01234567"`, the code will print `76543210`.
- C. If `/* SOME VALUE */` is `"012345678"`, the code will print `103254768`.
- D. If `/* SOME VALUE */` is `"012345678"`, the code will print `876543210`.
- E. If `/* SOME VALUE */` is `"012345678"`, the code will print `103254687`.
- F. If `/* SOME VALUE */` is `"012345678"`, the code will print less than 9 characters, because the terminating character `'\0'` is moved to somewhere in the middle of that string.

Solution: Be careful with the case `n == 3`.

(7) (3') [C] A student writes these functions to remove all digits from a C string:

```
void remove_one_char(char *cur) {
    while (*cur != '\0') {
        *cur = *(cur + 1);
```

```

        ++cur;
    }
}
void my_awesome_remove_digits(char *str) {
    while (*str != '\0') {
        if (isdigit(*str))
            remove_one_char(str);
        str++;
    }
}

```

```

char str[] = "0a12b3c4d";
my_awesome_remove_digits(str);

```

After the call to `my_awesome_remove_digits`, what is the value of `strlen(str)`?

- A. 4.
- B. 5.
- C. 9.
- D. This call to `my_awesome_remove_digits` will trigger a runtime error. Memory past the end of string `'\0'` is accessed.

Solution: To make that function correct, add an `else` before `str++`.

(8) (3') [C++] Select the pieces of code in which the call to `fun` compiles.

- A. `void fun(int a[100]);`
`int ival = 42;`
`fun(&ival);`
- B. `void fun(int a[100]);`
`int a[20];`
`fun(a);`
- C. `void fun(int (&a)[100]);`
`int a[100];`
`fun(&a);`
- D. `void fun(int (*a)[100]);`
`int a[100];`
`fun(*a);`

(9) (3') [C++] Read the following code.

```

std::string foo(const std::string &str) {
    std::string result(str.size(), '*');
    for (int i = 0, j = str.size() - 1; i <= j; i++, j--) {
        result[i] = str[j];
        result[j] = str[i];
    }
    return result;
}

std::string func(const std::string &str, const std::string &part) {
    std::size_t pos = str.find(foo(part));
}

```

```

    if (pos == std::string::npos) {
        return {};
    } else {
        return str.substr(0, pos)
            + foo(str.substr(pos, part.size()))
            + str.substr(pos + part.size(), str.size() - pos - part.size());
    }
}

```

```
std::cout << func("aabbcbcaab", "bc") << std::endl;
```

- A. `foo(str)` always returns a reversed `str`.
- B. The string returned by `foo(str)` may contain a '*' if the length of `str` is odd.
- C. The code prints `aabcbcaab`.
- D. The code prints `aabbbccaab`.

Solution: `foo(str)` always returns a reversed `str` even if the length of `str` is odd, because the loop condition is `i <= j`.
`func(str, part)` returns a copy of `str` with the first occurrence of reversed `part` reversed.

- (10) (3') [C++] Select the pieces of code in which the following range-based for loop can be used to traverse `a`.

```

for (const auto &x : a)
    do_something_with(x);

```

- A. `int a[100][{}];`
- B. `int *a = new int[100][{}];`
- C. `std::vector<std::string> a(10, "hello");`
- D. `std::string a = "world";`
- E. `void foo(int (&a)[10]) { /* ... */ }`
- F. `void foo(int a[10]) { /* ... */ }`

- (11) (3') [C++] Read the following code.

```

class Dynarray {
    int *m_storage;
    std::size_t m_length;
public:
    int &operator[](std::size_t n) const {
        return m_storage[n];
    }
};

```

Let `a` be a `Dynarray` and `ca` be a `const Dynarray`, and suppose both of them are non-empty. Which of the following is/are true?

- A. This `operator[]` function does not compile, because it is a `const` member function but returns a non-`const` reference to one of its members.
- B. `ca[0]` compiles.
- C. `a[0]` does not compile, because a `const` member function can only be called on a `const` object.

- D. `++ca[0]` compiles, and increments the element indexed 0 by 1.
- E. `++ca[0]` compiles, but the behavior is undefined.
- F. If we change the type of the member `m_storage` to `int[10]`, this code does not compile.

(12) (3') [C++] Read the following code.

```
int iarr[10]; // global
int main() {
    int *p1;
    int *p2 = new int[10];
    int *p3 = new int[10]{};
    int a[10] = {1};
    delete[] p2;
    delete[] p3;
}
```

- A. The elements in `iarr` are initialized to zero.
- B. `p1` is *default initialized*, whose value is the null pointer value.
- C. The values of `p2[0]`, `p2[1]`, ..., `p2[9]` are zero.
- D. The values of `p3[0]`, `p3[1]`, ..., `p3[9]` are zero.
- E. Every element in `a` is initialized to 1.

(13) (3') [C++] Now we want to design a `Book` class representing a book. This may be used in a bookstore management program, where each book has its title, the ISBN number and a price.

```
class Book {
public:
    Book(const std::string &title_, const std::string &isbn_, double price_); // (*)
private:
    const std::string title;
    const std::string isbn;
    double price = 0.0;
};
```

Which of the following statements is/are true?

- A. `Book` does not have a default constructor, since it has a user-declared constructor.
- B. If the member `price` does not appear in the initializer list of some constructor, it is initialized to 0.0 due to the in-class default member initializer.
- C. If the constructor (*) is defined as follows, `title` will be initialized after `price`.

```
// In class Book
Book(const std::string &title_, const std::string &isbn_, double price_)
    : price{price_}, isbn{isbn_}, title{title_} {}
```

- D. The constructor (*) can be defined as follows.

```
// In class Book
Book(const std::string &title_, const std::string &isbn_, double price_) {
    title = title_;
    isbn = isbn_;
    price = price_;
}
```


Solution: D. Since `title` and `isbn` are `const`, they must be initialized in the initializer list.

- (14) (3') [C++] Consider the `Book` class again, with more member functions added to it.

```
class Book {
public:
    Book(const std::string &title_, const std::string &isbn_, double price_);
    auto& get_title()      { return title;    }
    auto& get_isbn()       { return isbn;     }
    double total_price(int n) { return n * price; }
private:
    const std::string title;
    const std::string isbn;
    double price = 0.0;
};
```

Which of the following statements is/are true?

- A. The return type of `get_title` and `get_isbn` is `const std::string &`.
- B. The member functions `get_title`, `get_isbn` and `total_price` should be made `const`, because they do not modify the state of the object.
- C. `Book` has a copy constructor, a copy assignment operator, a move constructor, a move assignment operator and a destructor.
- D. The compiler will generate a destructor for `Book` as if it were defined as

```
// In class Book
~Book() {
    delete &price;
    delete &isbn;
    delete &title;
}
```

Solution: The copy constructor, the move constructor and the destructor of `Book` are implicitly declared, and implicitly defined if they are used. The copy assignment operator and the move assignment operator are implicitly declared and implicitly deleted since `title` and `isbn` are `const`.

The word "has" in choice C is not clear. Whether C is selected does not affect the score.

- (15) (3') [C++] Let the class `Book` be defined as above. Suppose we use a `std::vector<Book>` to store the books.

```
std::vector<Book> books;
```

Which of the following is/are true?

- A. `Book b("C++ Primer", "9780321714114", 47.99);`
`books.push_back(b);`
 This code appends a copy of `b` to the end of `books`.
- B. `void add_book(const std::string &title, const std::string &isbn, double price) {`
`Book b(title, isbn, price);`
`books.push_back(b);`
`}`

b is appended to the end of **books** through a move, not a copy.

C. `void add_book(const std::string &title, const std::string &isbn, double price) {
 books.push_back(Book(title, isbn, price));
}`

An object of type **Book** is constructed by `Book(title, isbn, price)`, and then moved into the vector.

D. `void add_book(const std::string &title, const std::string &isbn, double price) {
 books.emplace_back(title, isbn, price);
}`

The arguments (`title, isbn, price`) will be forwarded to the constructor of **Book**, so that an object of type **Book** is constructed directly in that vector without move or copy.

(16) (3') [C++] Suppose we have two classes **Item** and **DiscountedItem** defined as follows:

```
class Item {
public:
    Item(const std::string &name, double price)
        : m_name(name), m_price(price) {}
protected:
    std::string m_name;
    double m_price = 0.0;
};
```

```
class DiscountedItem : public Item {
public:
    DiscountedItem(const std::string &name, double price, double disc);
private:
    double m_discount = 1.0;
};
```

Which of the following is/are true?

A. The constructor of **DiscountedItem** can be defined as follows:

```
DiscountedItem(const std::string &name, double price, double disc)
    : m_name{name}, m_price{price}, m_discount{disc} {}
```

Because `m_name` and `m_price` are protected members that are accessible in **DiscountedItem**.

B. The constructor of **DiscountedItem** can be defined as follows:

```
DiscountedItem(const std::string &name, double price, double disc) {
    m_name = name;
    m_price = price;
    m_discount = disc;
}
```

C. If the member `m_name` of **Item** is private, it is still inherited by **DiscountedItem**.

D. `int main() {`

```
    std::shared_ptr<Item> dip = std::make_shared<DiscountedItem>("Coke", 3.0, 0.8);
}
```

The destructor of `dip` will call the destructor of **DiscountedItem** because `dip` is a smart pointer.

Solution: B. Since `Item` has a user-provided constructor, it does not have a default constructor. If the constructor of `Item` is not called explicitly in the constructor initializer list of `DiscountedItem`, the default constructor of `Item` is called, which results in an error.

(17) (3') [C++] Let `Item` and `DiscountedItem` be defined as follows.

```
class Item {
public:
    Item(const std::string &name, double price)
        : m_name(name), m_price(price) {}
    std::string to_string() const {
        return "Name: " + m_name + ", price: " + std::to_string(m_price);
    }
    void set_price(double price) { m_price = price; }
    virtual ~Item() = default;
protected:
    std::string m_name;
    double m_price = 0.0;
};

class DiscountedItem : public Item {
public:
    DiscountedItem(const std::string &name, double price, double disc)
        : Item(name, price), m_discount(disc) {}
    void set_discount(double disc) { m_discount = disc; }
private:
    double m_discount = 1.0;
};
```

Consider the following code.

```
std::vector<std::unique_ptr<Item>> snacks;
snacks.push_back(std::make_unique<Item>("cookies", 7.5));
snacks.push_back(std::make_unique<DiscountedItem>("chips", 6.0, 0.6));
snacks[0]->set_discount(0.5);           // (1)
snacks[1]->set_discount(0.8);           // (2)
snacks[1]->set_price(8.0);               // (3)
std::cout << *snacks[0] << std::endl;   // (4)
std::cout << std::string(*snacks[0]) << std::endl; // (5)
```

Which of the following explanations is/are correct?

- A. Line (1) does not compile, because `Item` has no member `set_discount()`.
- B. Line (2) compiles, because `snacks[1]` points to a `DiscountedItem`.
- C. Line (3) does not compile, because `DiscountedItem` does not have an implementation of `set_price()`.
- D. Line (4) does not compile, because there is no `operator<<` defined with operands `std::ostream&` and `Item`.
- E. Line (5) compiles, because `Item` has a member function `to_string()` that will be automatically used as a conversion to `std::string`.

- (18) (3') [C++] Let `Item` and `DiscountedItem` be defined as in question (17). Now we want to add a group of functions `net_price(n)`, which returns the net price of `n` items. The following function should print the correct net price according to the dynamic type of `item`.

```
void print_net_price(const Item &item, int n) {
    std::cout << "net price: " << item.net_price(n) << std::endl;
}
```

Which of the following definitions is/are correct?

- A. *// In class Item*
`double net_price(int n) const { return n * m_price; }`
// In class DiscountedItem
`double net_price(int n) const { return n * m_price * m_discount; }`
- B. *// In class Item*
`virtual double net_price(int n) const { return n * m_price; }`
// In class DiscountedItem
`double net_price(int n) const { return n * m_price * m_discount; }`
- C. *// In class Item*
`virtual double net_price(int n) const { return n * m_price; }`
// In class DiscountedItem
`double net_price(int n) { return n * m_price * m_discount; }`
- D. *// In class Item*
`double net_price(int n) const { return n * m_price; }`
// In class DiscountedItem
`virtual double net_price(int n) const override { return n * m_price * m_discount; }`

- (19) (3') [C++] Consider the following class representing a complex number $a + bi$, where $a, b \in \mathbb{R}$.

```
class Complex {
    double real;
    double imaginary;
public:
    Complex(double a, double b = 0) : real(a), imaginary(b) {} // (1)
    Complex operator-(const Complex &x) const; // (2)
    Complex operator*(const Complex &x) const {
        return {
            real * x.real - imaginary * x.imaginary,
            real * x.imaginary + x.real * imaginary
        };
    }
    friend Complex operator+(const Complex &, const Complex &); // (3)
};
Complex operator+(const Complex &lhs, const Complex &rhs) {
    return {
        lhs.real + rhs.real,
        lhs.imaginary + rhs.imaginary
    };
}
```

Let `z` be an object of type `Complex`. Which of the following is/are true?

- A. The function (2) is the unary minus operator (`-x`), because it only accepts one argument.

- B. `0 + z` compiles, while `0 * z` does not compile.
- C. If the function (1) is **explicit**, the expression `0 + z` does not compile.
- D. The function (3) is a member of **Complex**.
- (20) (3') [C++] Consider the class **Complex** defined as above. Now we want to define **operator+=** for it. Select the **unique best** implementation of **operator+=** which compiles, involves no undefined behaviors and adheres to the conventions.
- A. *// In class Complex*
`Complex &operator+=(Complex &rhs) {`
 `real += rhs.real;`
 `imaginary += rhs.imaginary;`
 `return *this;`
`}`
- B. *// In class Complex*
`auto &operator+=(const Complex &rhs) {`
 `real += rhs.real;`
 `imaginary += rhs.imaginary;`
 `return *this;`
`}`
- C. *// In class Complex*
`void operator+=(const Complex &rhs) {`
 `real += rhs.real;`
 `imaginary += rhs.imaginary;`
`}`
- D. *// In class Complex*
`Complex &operator+=(const Complex &rhs) {`
 `real += rhs.real;`
 `imaginary += rhs.imaginary;`
`}`
- E. *// In class Complex*
`Complex &operator+=(const Complex &rhs) {`
 `real += rhs.real;`
 `imaginary += rhs.imaginary;`
 `return this;`
`}`

2. (15 points) Single Peak

We call a series of numbers **Single-Peaked** if there is one and only one “peak” such that:

- “Peak” is the **only** greatest number (high peak) / smallest number (low peak), and cannot be the first or last in the series.
- Numbers before the “peak” are non-descending (high peak) / non-ascending (low peak).
- Numbers after the “peak” are non-ascending (high peak) / non-descending (low peak).

Write a function named `isSinglePeaked`, that takes a parameter of `const std::vector<int>&`, and returns `true` if its contents are single-peaked, and `false` otherwise. The behavior is undefined if the vector passed in contains less than three elements.

Here are some examples of how your function should behave. Variables with “true” in names should evaluate to `true`, and similarly for `false`.

```
bool true1  = isSinglePeaked({1, 3, 5, 4, 2});
bool true2  = isSinglePeaked({8, 2, -2, -5, 6});
bool true3  = isSinglePeaked({1, 3, 3, 4, 6, 4, 4});
bool false1 = isSinglePeaked({5, 4, 3, 2, 1});
bool false2 = isSinglePeaked({6, 0, 0, 2});
```

Write your function here:

Solution:

```
auto checkPeak(const std::vector<int> &v) {
    std::size_t i = 0;
    while (i + 1 < v.size() && v[i] <= v[i + 1])
        ++i;
    auto j = v.size() - 1;
    while (j > 0 && v[j] <= v[j - 1])
        --j;
    return i > 0 && i + 1 < v.size() && i == j;
}

auto negate(std::vector<int> v) {
    for (auto &x : v)
        x = -x;
    return v;
}

bool isSinglePeaked(const std::vector<int> &v) {
    return checkPeak(v) || checkPeak(negate(v));
}
```

3. (15 points) Time and Clocks

Suppose you are coding three classes, `Time`, `Clock`, and `AlarmClock`.

- `Time` stores the hour and minute of a certain time moment.
- `Clock` has a `Time`, and can **display** the time or **tick** to the next minute.

- AlarmClock inherits from Clock, and saves another Time for alarming.

Here is some incomplete code of these classes. Complete it according to the instructions.

```
class Time {
public:
    Time(int minutes = 0)
        : m_hour((minute / 60) % 24), m_minute(minute % 60) {}
    Time(int hour, int minute)
        : m_hour((hour + minute / 60) % 24), m_minute(minute % 60) {}
    // (1)
private:
    int m_hour;
    int m_minute;
    static std::string fill2(int x) {
        return x < 10 ? "0" + std::to_string(x) : std::to_string(x);
    }
};

class Clock {
public:
    Clock(Time time) : m_time(time) {}
    void tick() {
        ++m_time; // It should add 1 minute to m_time.
    }
    virtual void display() const {
        // It should print something like "Now: 09:03".
        std::cout << "Now: " << m_time << std::endl;
    }
    // (*)
protected:
    Time m_time;
};

class AlarmClock : public Clock {
public:
    AlarmClock(Time time, Time alarm)
        : Clock(time), m_alarm(alarm) {}
    // (2)
private:
    Time m_alarm;
};
// (3)
```

- (1) (3') **Clock** should have a destructor at line (*). Write the destructor so that no undefined behavior happens when **Clock** and **AlarmClock** are used in polymorphism.

Solution:

```
virtual ~Clock() = default;
```

Also correct:

```
virtual ~Clock() {}
```

- (2) (6') Currently the code does not compile, because **Time** lacks some operators. Add code to (1), (3), or both places, so that the code compiles and behaves like stated in the comments. Specify which of your code goes to (1) or (3). You may find the **Time::fill2** function useful.

Solution: (1):

```
Time &operator++() {  
    ++m_minute;  
    if (m_minute == 60) {  
        m_hour = (m_hour + 1) % 24;  
        m_minute = 0;  
    }  
    return *this;  
}  
friend std::ostream &operator<<(std::ostream &, const Time &);
```

(3):

```
friend std::ostream &operator<<(std::ostream &os, const Time &t) {  
    return os << Time::fill2(t.m_hour) << ":" << Time::fill2(t.m_minute);  
}
```

This friend function `operator<<` can also be defined at (1). `Time::fill2` can be replaced with `t.fill2`.

(3) (6') Finally, **AlarmClock** should override **display()**, in the following manner:

- It should print 2 or 3 lines.
- The first line should look like **Now: 09:03**, identical to what **Clock::display()** prints.
- The second line should look like **Alarm: 09:15**, printing the value of **m_alarm**.
- If the time now is its alarm time, print **ALARM!** on the third line.

Add code to do so. Feel free to add any other functions. Any of your code can go to (1), (2), or (3), and you should specify where it goes.

Solution: (1):

```
bool operator==(const Time &rhs) const {  
    return m_hour == rhs.m_hour && m_minute == rhs.m_minute;  
}
```

(2):

```
void display() const override {  
    Clock::display();  
    std::cout << "Alarm: " << m_alarm << std::endl;  
    if (m_alarm == m_time)  
        std::cout << "ALARM!" << std::endl;  
}
```

Other ways to enable **AlarmClock::display** to compare **m_alarm** with **m_time** are allowed. For example, declare **AlarmClock** as a friend class of **Time** in (1), so that **m_hour** and **m_minute** can be accessed directly.

The first line can also be printed directly using

```
std::cout << "Now: " << m_time << std::endl;
```

4. (10 points) Ref-qualified member functions

Note: This problem is actually easier than the previous one.

Apart from the `const` qualification, a member function can also be *ref-qualified*. The syntax of ref-qualified member functions is as follows.

- (1) `return_type function_name(parameter_list) constoptional & noexceptoptional;`
- (2) `return_type function_name(parameter_list) constoptional && noexceptoptional;`

Explanation:

- (1) *lvalue ref-qualified* member function of a class `X`: The implicit object parameter has type `X &` (or `const X &`, if it is a `const` member function).
- (2) *rvalue ref-qualified* member function of a class `X`: The implicit object parameter has type `X &&` (or `const X &&`, if it is a `const` member function).

For example:

```
#include <iostream>
struct X {
    void foo() const & { std::cout << "lvalue reference-to-const" << std::endl; }
    void foo()      && { std::cout << "rvalue reference" << std::endl; }
};
int main() {
    X x;
    x.foo();           // prints "lvalue reference-to-const"
    const X &cx = x;
    cx.foo();          // prints "lvalue reference-to-const"
    std::move(x).foo(); // prints "rvalue reference"
}
```

The member functions `foo` in the example above can be seen as

```
void X_member_foo(const X &self) {
    std::cout << "lvalue reference-to-const" << std::endl;
}
void X_member_foo(X &&self) {
    std::cout << "rvalue reference" << std::endl;
}
```

and the calls to `foo` can be seen as

```
int main() {
    X x;
    X_member_foo(x);           // matches "const X &self"
    const X &cx = x;
    X_member_foo(cx);          // matches "const X &self"
    X_member_foo(std::move(x)); // matches "X &&self"
}
```

Note: unlike **const** qualification, *ref-qualification* does not change the type and properties of the **this** pointer: the type of **this** is still **X *** (or **const X *** if it is a **const** member function), and ***this** is always an lvalue expression.

The ref-qualification allows us to define different versions of a member function for lvalues and rvalues. Now consider the **Dynarray** class representing a dynamic array:

```
class Dynarray {
    int *m_storage;
    std::size_t m_length;
public:
    Dynarray sorted() const {
        Dynarray ret = *this;
        std::sort(ret.m_storage, ret.m_storage + ret.m_length);
        return ret;
    }
    // some other members ...
};
```

The member function **sorted()** returns a copy of ***this** but with all elements sorted in ascending order. However, if **sorted()** is called on a non-**const** rvalue (e.g. **std::move(a).sorted()**, or **Dynarray(begin, end).sorted()**), there is no need to copy the original array - we can directly sort the elements in ***this**, and return an rvalue reference to ***this** (obtained by **std::move**).

Use the **const**- and ref-qualifications of member functions to achieve this. Fill in the blanks for the return types and qualifications, and complete the function bodies.

```
class Dynarray {
    int *m_storage;
    std::size_t m_length;
public:
    _____ sorted() _____ {
        // YOUR CODE HERE ...

    }

    _____ sorted() _____ {
        // YOUR CODE HERE ...

    }
    // some other members ...
};
```

Solution:

```
class Dynarray {
    int *m_storage;
    std::size_t m_length;
public:
    Dynarray sorted() const & {
        auto ret = *this;
        std::sort(ret.m_storage, ret.m_storage + ret.m_length);
        return ret;
    }
    Dynarray &&sorted() && {
        std::sort(m_storage, m_storage + m_length);
        return std::move(*this);
    }
    // some other members ...
};
```

The return type of the second function was required to be `Dynarray &&` (which yields an **xvalue**), but it is also ok to use `Dynarray` (which yields a **prvalue**). But the returned expression must be `std::move(*this)`.

CS100 Computer Programming

Quiz 1

April 19, 2023

Answer the questions according to the C++17 standard.

For the compiler-generated special member functions, ignore whether they are **constexpr**, and ignore whether they are **noexcept** except for move operations.

1. (15 points) Your name: _____. Your student ID: _____.
Your email: _____@shanghaitech.edu.cn
2. (10 points) Select the pieces of code that have (or may lead to) undefined behaviors.

- A. `std::vector<double> vec;`
`for (std::size_t i = 0; i != n; ++i)`
`std::cin >> vec[i];`
- B. `void extend(std::vector<double> &vec) {`
`for (auto x : vec)`
`vec.push_back(x);`
`}`
- C. `std::size_t npos = -1;`
- D. `int main() {`
`std::string str;`
`std::cout << str << std::endl;`
`}`
- E. `int *ptr = nullptr;`
`delete ptr;`

3. (10 points) Let `ival` be an `int`, and let `ptr` be of some pointer type. Select the expressions that yield an **rvalue**.

A. `++ival` B. `ival++` C. `*&ptr` D. `&*ptr` E. `ptr[ival]` F. `*(ptr + ival)`

4. (10 points) Let class `X` be defined as follows.

```
struct X {  
    int a, b;  
    std::string s;  
    X() = default;  
    X(X &&) = default;  
    ~X() { std::cout << "Goodbye world" << std::endl; }  
};
```

Which of the following statements are true?

- A. In the destructor `~X`, the destructor of `std::string` is called to destroy the member `s` before "Goodbye world" is printed.

B. The compiler will generate a default constructor for `X` (if it is used) which default-initializes all the members.

C. The compiler will generate a move constructor for `X` (if it is used) as if it were defined as

```
X(X &&other) noexcept
    : a(std::move(other.a)), b(std::move(other.b)), s(std::move(other.s)) {}
```

D. The compiler will generate a move constructor for `X` (if it is used) as if it were defined as

```
X(X &&other) noexcept
    : a(other.a), b(other.b), s(other.s) {}
```

There is no need to apply `std::move` to the members of `other`, because `other` is an rvalue.

5. (10 points) Suppose `Dynarray` has both a copy assignment operator and a move assignment operator. Select the situation(s) where the **copy assignment operator** of `Dynarray` is used.

A.

```
Dynarray concat(const Dynarray &a, const Dynarray &b) {
    Dynarray result(a.size() + b.size());
    // Concatenates the contents of `a` and `b`. Details are omitted.
    return result;
}
int main() {
    Dynarray a, b;
    a = concat(a, b);
}
```

B. `Dynarray a; Dynarray b = a;`

C. `Dynarray *a, *b; a = b;`

D. `std::vector<Dynarray> vec(10); Dynarray a; vec[0] = a;`

6. (10 points) Which of the following statements regarding **const member functions** is/are true?

A. **const** member functions can only be called on **const** objects.

B. **const** member functions cannot call non-**const** member functions (without a `const_cast`).

C. In a **const** member function of class `X`, the implicit `this` pointer has type `const X *`.

D. If a non-**const** member function does not modify any data member, the compiler will make it a **const** member function.

7. (10 points) Select the situation(s) where the **default constructor** of the class `X` is used.

A. `X a[100];` B. `auto p = new X;` C. `X a();` D. `void fun(X a) {}`

8. (15 points) For each piece of code, write down the type of `var`.

(a) `auto ival = 42; auto *var = &ival;`

(a) _____

(b) `std::vector v(10, 3.14);`
`auto var = v[0];`

(b) _____

(c) `std::vector<std::vector<std::string>> vvs;`
`for (const auto &vs : vvs)`
`for (const auto &var : vs)`
`do_something(var);`

(c) _____

CS100 Introduction to Programming
Fall 2023
Midterm Exam

Instructors: Ying Cao

Time: Dec 19th 13:00-14:40

INSTRUCTIONS

Please read and follow the following instructions:

- You have 100 minutes to answer the questions.
- You are not allowed to bring any electronic devices including regular calculators.
- You are not allowed to discuss or share anything with others during the exam.
- You should write the answer to every problem in the dedicated box **clearly**.
- You should write **your name and your student ID** as indicated on the top of **each page** of the exam sheet.

Name	
Student ID	

Please write your answers to the multiple choices questions in the following table.

(1)	(2)	(3)	(4)	(5)
AD	AD	any	B	C
(6)	(7)	(8)	(9)	(10)
ACD	ACD	ABD	CD	AB
(11)	(12)	(13)	(14)	(15)
C	BC	BD	CD	D

1. (75 points) Multiple Choices

Each of the following questions has **one or more** correct choices.

You will get some proportion of a question's points if you choose a non-empty proper subset of its correct choices.

The questions marked "[C]" are based on the C17 standard (ISO/IEC 9899:2018). The questions marked "[C++]" are based on the C++17 standard (ISO/IEC 14882:2017).

- (1) (5') [C] Read the following code. Select the correct statement(s).

```
#define N 100
```

```
int a[N], b[N]; // (1)
```

```
void plus(int *a, int *b, int n) { // (2)
    for (int i = 0; i < n; ++i) // (3)
        a[i] += b[i];
}
```

```
void minus(int *a, int *b, int n) {
    for (int i = 0; i < n; ++i) // (4)
        a[i] -= b[i];
}
```

- A. The line (1) will be rewritten as `int a[100], b[100];` by the preprocessor.
 - B. The identifier `a` at (1) and the one at (2) are the same variable.
 - C. The identifier `i` at (3) and the one at (4) are the same variable.
 - D. The elements in `a` at (1) are initialized to zero.
- (2) (5') [C] The following function accepts a string and tests whether it is a palindrome.

```
int is_palindrome(char *str) {
    size_t n = strlen(str);
    for (size_t i = 0, j = n - 1; i < j; ++i, --j)
        if (str[i] != str[j])
            return 0;
    return 1;
}
```

Select the correct statement(s).

- A. Since `is_palindrome` does not modify the given string, the type of the parameter `str` should be `const char *`.
 - B. Since `sizeof(char) == 1`, `strlen(str)` can be replaced with `sizeof(str)`.
 - C. `is_palindrome("()")` returns 1.
 - D. The code involves undefined behavior if `str` is an empty string.
- (3) (5') [C] Suppose we have defined the following `struct` represents a vector in linear algebra.

```
struct Vector {
    size_t dim;
    double *data;
};
```

Select the correct statement(s).

- A. The following function sets a given `Vector` to empty.
- ```
void vector_init(struct Vector vec) {
 vec = (struct Vector){.dim = 0, .data = NULL};
}
```
- To use this function to set `v` to empty, we can write `vector_init(v);`.
- B. The following function sets a given `Vector` to  $(0, \dots, 0) \in \mathbb{R}^n$ .
- ```
void vector_init(struct Vector *vec, size_t n) {
    vec = (struct Vector){.dim = n, .data = malloc(sizeof(double) * n)};
}
```
- To use this function to set `v` to $(0, \dots, 0) \in \mathbb{R}^n$, we can write `vector_init(&v, n);`.
- C. The following function returns the sum of two `Vectors`.
- ```
struct Vector *vector_add(const struct Vector *lhs, const struct Vector *rhs) {
 size_t dim = lhs->dim < rhs->dim ? rhs->dim : lhs->dim;
 struct Vector result = {.dim = dim, .data = calloc(dim, sizeof(double))};
 for (size_t i = 0; i < lhs->dim; ++i)
 result.data[i] += lhs->data[i];
 for (size_t i = 0; i < rhs->dim; ++i)
 result.data[i] += rhs->data[i];
 return &result;
}
```
- D. `free(v)`, where `v` is of type `struct Vector`, will call `free(v.data)` automatically to release the memory it has allocated.

**Solution:** None of the choices are correct.

- (4) (5') [C] Which of the following statements is/are true?
- A. Suppose `sizeof(int) == 4` and `sizeof(long long) == 8`.
- ```
int ival = 10000000;
long long llval = 1ll * ival * ival;
```
- This code has undefined behavior because `ival * ival` overflows.
- B. Suppose `i` is of type `int`. `printf("%d%d\n", i, i++)` has undefined behavior.
- C. Suppose `f` is a `float`. `printf("%d", f)` has the same effect as `printf("%d", (int)f)`. For example, `printf("%d", f)` prints "3" if the value of `f` is 3.14.
- D. `char buffer[100];`
`scanf("%s", buffer);`
 If the input is longer than 100 characters, `scanf` will allocate a larger block of memory for `buffer` automatically.
- (5) (5') [C++] Let `a` be of type `int [9][10]`. Select the pieces of code that makes `foo(a)` compile.
- A. `void foo(int **a);`
 B. `void foo(int (*a)[9]);`
 C. `void foo(int (&a)[9][10]);`
 D. `void foo(int (&a)[10][9]);`
- (6) (5') [C++] We want to use a nested `vector` to represent a matrix. Which of the following statements is/are true?
- A. `std::vector<std::vector<double>>` is a valid type.
 B. `std::vector<std::vector<double>> matrix(n, m);`

`matrix` is initialized to be with `n` rows and `m` columns. That is, `matrix` is a vector that contains `n` elements, each of which is a vector containing `m` doubles.

C. `std::vector matrix(n, std::vector(m, 0.0));`

The type of `matrix` is deduced to be `std::vector<std::vector<double>>`.

D. `std::vector matrix(n, std::vector(m, 0.0));`

`matrix` is initialized to be with `n` rows and `m` columns, with each element initialized to `0.0`.

- (7) (5') [C++] Select the pieces of code in which the following range-based for loop can be used to traverse `a`.

```
for (const auto &x : a)
    do_something_with(x);
```

A. `int a[100][{}];`

B. `int *a = new int[100][{}];`

C. `std::vector<std::string> a(10, "hello");`

D. `std::string a = "world";`

- (8) (5') [C++] Consider the class example `Dynarray` (which is also in Homework 5). Suppose its data members are defined as follows.

```
class Dynarray {
    int *m_storage;
    std::size_t m_length;
};
```

Select the correct statement(s). For choices C and D, suppose we want to add the member functions `begin` and `end` so that a `Dynarray` can be traversed using a range-based for loop.

A. `m_length` is private.

B. In the following member function, the expression `m_storage` has type `int *const`, because `this` has type `const Dynarray *`.

```
class Dynarray {
public:
    int operator[](std::size_t n) const { return m_storage[n]; }
};
```

C. The following is a good design for `begin` and `end`.

```
class Dynarray {
public:
    int *begin() { return m_storage; }
    int *end() { return m_storage + m_length; }
};
```

D. The following is a good design for `begin` and `end`.

```
class Dynarray {
public:
    int* begin() { return m_storage; }
    int* end() { return m_storage + m_length; }
    const int* begin() const { return m_storage; }
    const int* end() const { return m_storage + m_length; }
};
```

- (9) (5') [C++] Now we want to design a `Book` class representing a book. This may be used in a bookstore management program, where each book has its title, the ISBN number and a price.

```

class Book {
public:
    Book(const std::string &title_, const std::string &isbn_, double price_); // (*)
    Book() = default;

private:
    std::string title;
    std::string isbn;
    double price = 0.0;
};

```

Which of the following statements is/are true?

- A. `Book` does not have a default constructor, since it has a user-declared constructor.
- B. If the constructor (*) is defined as follows, `title` will be initialized after `price`.
`Book::Book(const std::string &title_, const std::string &isbn_, double price_) : price{price_}, isbn{isbn_}, title{title_} {}`
- C. The compiler generates a default constructor for `Book`, in which the member `price` is initialized to `0.0`.
- D. Since the move operations of `std::string` are cheap, we can rewrite the constructor (*) as
`Book::Book(std::string title_, std::string isbn_, double price_) : title{std::move(title_)}, isbn{std::move(isbn_)}, price{price_} {}`
so that rvalue string arguments are moved, not copied.

(10) (5') [C++] Consider the `Book` class again, with more member functions added to it.

```

class Book {
public:
    const std::string &get_title() { return title; }
    double total_price(int n) { return n * price; }
    bool operator==(const Book &rhs) const { return isbn == rhs.isbn; }
    bool operator!=(const Book &) const;
    // other members ...

private:
    std::string title;
    std::string isbn;
    double price = 0.0;
};

```

Which of the following statements is/are true?

- A. The member functions `get_title` and `total_price` should be `const` member functions, because they do not modify the state of the object.
- B. Let `b1` and `b2` be two `Books`. The expression `b1 == b2` is effectively `b1.operator==(b2)`, where the parameter `rhs` is bound to `b2`.
- C. The member function `operator==` does not compile, because it attempts to access `rhs.isbn` which is a private member of `rhs`.
- D. The following is a reasonable design for the inequality operator.
`bool Book::operator!=(const Book &rhs) const { return title != rhs.title; }`

(11) (5') [C++] The function `create` in the following code is a *factory function* that creates an object dynamically and returns a smart pointer to it.

```

class Book {
private:
    Book(std::string t, std::string i, double p)
        : title{std::move(t)}, isbn{std::move(i)}, price{p} {}
    Book(const Book &) = default;
    Book(Book &&) = default;

public:
    static auto create(std::string title, std::string isbn, double price) {
        return std::unique_ptr(new Book(std::move(title), std::move(isbn), price));
    }
    // other members ... but with no public constructors
private:
    // data members as before
};

```

Select the correct statement(s).

- A. In the function `create`, the `this` pointer has type `Book *`.
- B. The return type of the function `create` is `std::unique_ptr`.
- C. The only way for the user to create a `Book` is to call the factory function `create`, e.g. `Book::create("C++ Primer", "9780321714114", 75)`.
- D. The function `create` can be rewritten as

```

// In class Book
static auto create(std::string title, std::string isbn, double price) {
    return std::make_unique<Book>(std::move(title), std::move(isbn), price);
}

```

Solution: The `new` expression here cannot be replaced with `std::make_unique`, because `std::make_unique` cannot access the private constructors.

- (12) (5') [C++] Consider the following class representing a complex number $a + bi$, where $a, b \in \mathbb{R}$.

```

class Complex {
    double real;
    double imaginary;
public:
    Complex(double x) : real(x), imaginary(0) {} // (1)
    Complex(double a, double b) : real(a), imaginary(b) {}
    Complex operator-(const Complex &x) const; // (2)
    Complex operator*(const Complex &x) const {
        return {
            real * x.real - imaginary * x.imaginary,
            real * x.imaginary + x.real * imaginary
        };
    }
    friend Complex operator+(const Complex &, const Complex &); // (3)
};
Complex operator+(const Complex &lhs, const Complex &rhs) {

```

```
    return {lhs.real + rhs.real, lhs.imaginary + rhs.imaginary};
}
```

Let `z` be an object of type `Complex`. Which of the following is/are true?

- A. The function (2) is the unary minus operator (`-x`), because it only accepts one argument.
 - B. `0 + z` compiles, while `0 * z` does not compile.
 - C. If the function (1) is `explicit`, the expression `0 + z` does not compile.
 - D. The function (3) is a member of `Complex`.
- (13) (5') The standard library has a function `std::copy_if`, which is similar to `std::copy` but accepts one more parameter `pred` that is a unary predicate. Only the elements for which `pred` returns true will be copied. Now we want to copy the integers less than a threshold `k` from a `std::vector<int>` into a new vector. Select the correct implementations.
- A.

```
std::vector<int> work(const std::vector<int> &v, int k) {
    std::vector<int> result;
    std::copy_if(v.begin(), v.end(), result.begin(), [k](int x) { return x < k; });
    return result;
}
```
 - B.

```
struct LessThanK {
    int k;
    LessThanK(int k_) : k{k_} {}
    bool operator()(int x) const { return x < k; }
};
std::vector<int> work(const std::vector<int> &v, int k) {
    std::vector<int> result(v.size());
    std::copy_if(v.begin(), v.end(), result.begin(), LessThanK{k});
    return result;
}
```
 - C.

```
bool less_than_k(int x, int k) {
    return x < k;
}
std::vector<int> work(const std::vector<int> &v, int k) {
    std::vector<int> result(v.size());
    std::copy_if(v.begin(), v.end(), result.begin(), less_than_k);
    return result;
}
```
 - D.

```
std::vector<int> work(const std::vector<int> &v, int k) {
    std::vector<int> result(v.size());
    std::copy_if(v.begin(), v.end(), result.begin(), [k](int x) { return x < k; });
    return result;
}
```

- (14) (5') [C++] Suppose we have two classes `Item` and `DiscountedItem` defined as follows:

```
class Item {
public:
    Item(std::string name, double price) : m_name(std::move(name)), m_price(price) {}
protected:
    std::string m_name;
    double m_price = 0.0;
```

```
};
class DiscountedItem : public Item {
public:
    DiscountedItem(std::string name, double price, double disc); // (*)
private:
    double m_discount = 1.0;
};
```

Which of the following is/are true?

- A. The private members of `Item`, if any, are not inherited by `DiscountedItem`.
- B. The compiler generates a copy constructor for `DiscountedItem` as if it were defined as

```
// In class DiscountedItem
DiscountedItem(const DiscountedItem &other)
    : m_name(other.m_name), m_price(other.m_price), m_discount(other.m_discount) {}
```
- C. The destructor of `DiscountedItem` will invoke the destructor of `Item` to destroy the base class subobject.
- D. The constructor (*) can be defined as

```
DiscountedItem(std::string name, double price, double disc)
    : Item(std::move(name), price), m_discount(disc) {}
```

- (15) (5') [C++] Let `Item` and `DiscountedItem` be defined as in question (14). Now we want to add a group of functions `net_price(n)`, which returns the net price of `n` items. The following function should print the correct net price according to the dynamic type of `item`.

```
void print_net_price(const Item &item, int n) {
    std::cout << "net price: " << item.net_price(n) << std::endl;
}
```

Select the one best way of defining `net_price`.

- A.

```
// In class Item
double net_price(int n) const { return n * m_price; }
// In class DiscountedItem
double net_price(int n) const { return n * m_price * m_discount; }
```
- B.

```
// In class Item
virtual double net_price(int n) const { return n * m_price; }
// In class DiscountedItem
double net_price(int n) const { return n * m_price * m_discount; }
```
- C.

```
// In class Item
double net_price(int n) const { return n * m_price; }
// In class DiscountedItem
virtual double net_price(int n) const { return n * m_price * m_discount; }
```
- D.

```
// In class Item
virtual double net_price(int n) const { return n * m_price; }
// In class DiscountedItem
double net_price(int n) const override { return n * m_price * m_discount; }
```

Name:

ID:

2. (15 points) The “is-a” relationship

Public inheritance models the “is-a” relationship. Explain your understanding on this. Give some good and bad examples.

Solution: Public inheritance means “is-a”. Everything that applies to base classes must also apply to derived classes, because every derived class object is a base class object. Anywhere an object of type **Base** can be used, an object of type **Derived** can be used just as well.

Good example: A student is a person, a rectangle is a shape, etc.

Bad example: A square is a rectangle, but not everything applicable to a rectangle can be applied to a square.

3. (10 points) Ref-qualified member functions

Apart from the `const` qualification, a member function can also be *ref-qualified*. The syntax of ref-qualified member functions is as follows.

- (1) `return_type function_name(parameter_list) constoptional & noexceptoptional;`
- (2) `return_type function_name(parameter_list) constoptional && noexceptoptional;`

Explanation:

- (1) *lvalue ref-qualified* member function of a class `X`: The implicit object parameter has type `X &` (or `const X &`, if it is a `const` member function).
- (2) *rvalue ref-qualified* member function of a class `X`: The implicit object parameter has type `X &&` (or `const X &&`, if it is a `const` member function).

For example:

```
#include <iostream>
struct X {
    void foo() const & { std::cout << "lvalue reference-to-const" << std::endl; }
    void foo()      && { std::cout << "rvalue reference" << std::endl; }
};
int main() {
    X x;
    x.foo();           // prints "lvalue reference-to-const"
    const X &cx = x;
    cx.foo();          // prints "lvalue reference-to-const"
    std::move(x).foo(); // prints "rvalue reference"
}
```

The member functions `foo` in the example above can be seen as

```
void X_member_foo(const X &self) {
    std::cout << "lvalue reference-to-const" << std::endl;
}
void X_member_foo(X &&self) {
    std::cout << "rvalue reference" << std::endl;
}
```

and the calls to `foo` can be seen as

```
int main() {
    X x;
    X_member_foo(x);           // matches "const X &self"
    const X &cx = x;
    X_member_foo(cx);          // matches "const X &self"
    X_member_foo(std::move(x)); // matches "X &&self"
}
```

Note: unlike `const` qualification, *ref-qualification* does not change the type and properties of the `this` pointer: the type of `this` is still `X *` (or `const X *` if it is a `const` member function), and `*this` is always an lvalue expression.

The ref-qualification allows us to define different versions of a member function for lvalues and rvalues. Now consider the `Dynarray` class representing a dynamic array:

```
class Dynarray {
    int *m_storage;
    std::size_t m_length;
public:
    Dynarray sorted() const {
        Dynarray ret = *this;
        std::sort(ret.m_storage, ret.m_storage + ret.m_length);
        return ret;
    }
    // some other members ...
};
```

The member function `sorted()` returns a copy of `*this` but with all elements sorted in ascending order. However, if `sorted()` is called on a non-`const` rvalue (e.g. `std::move(a).sorted()`, or `Dynarray(begin, end).sorted()`), there is no need to copy the original array - we can directly sort the elements in `*this`, and return an rvalue reference to `*this` (obtained by `std::move`).

Use the `const`- and ref-qualifications of member functions to achieve this. Fill in the blanks for the return types and qualifications, and complete the function bodies.

Solution:

```
class Dynarray {
    int *m_storage;
    std::size_t m_length;
public:
    Dynarray sorted() const & {
        auto ret = *this;
        std::sort(ret.m_storage, ret.m_storage + ret.m_length);
        return ret;
    }
    Dynarray &&sorted() && {
        std::sort(m_storage, m_storage + m_length);
        return std::move(*this);
    }
    // some other members ...
};
```

The return type of the second function was required to be `Dynarray &&` (which yields an **xvalue**), but it is also ok to use `Dynarray` (which yields a **prvalue**). But the returned expression must be `std::move(*this)`.

CS100 Spring 2024

Quiz 1

Apr 24, 2024

1. (15 points) Your name: _____. Your student ID: _____.
Your email: _____@shanghaitech.edu.cn
2. (40 points) (C) The following functions are all related to “removing digits from a string”. For each function, does it correctly implement the behavior described in its documentation (the comments before it)? If not, what is wrong with it?

- (a) */// @brief Writes a string to 'dest' obtained from 'source' by removing all the digits.*
/// @param dest Points to a block of memory large enough to hold the result.
/// @param source Points to a null-terminated byte string.
- ```
void remove_digits(char *dest, const char *source) {
 while (*source != '\0') {
 if (!isdigit(*source))
 *dest++ = *source;
 ++source;
 }
}
```

**Solution:** Incorrect. It forgets to put a '\0' at the end.

- (b) */// @brief Writes a string to 'dest' obtained from 'source' by removing all the digits.*  
*/// @param dest Points to a block of memory large enough to hold the result.*  
*/// @param source Points to a null-terminated byte string.*
- ```
void remove_digits(char *dest, const char *source) {  
    strcpy(dest, source);  
    for (char *i = dest; *i != '\0';) {  
        if (isdigit(*i)) {  
            for (char *j = i; *j != '\0'; ++j)  
                *j = *(j + 1);  
        } else  
            ++i;  
    }  
}
```

Solution: Incorrect. The memory block pointed to by **dest** is only required to be large enough to hold the result. But the implementation actually requires more, since it copies the entire string **source** to it in the beginning.

- (c) */// @brief Writes a string to 'dest' obtained from 'source' by removing all the digits.
 /// @param dest Points to a block of memory large enough to hold the result.
 /// @param source Points to a null-terminated byte string.*

```
void remove_digits(char *dest, const char *source) {
    int cnt = 0;
    for (const char *i = source; *i != '\0'; ++i)
        if (!isdigit(*i))
            ++cnt;
    dest = calloc(cnt + 1, 1);
    for (const char *i = source; *i != '\0'; ++i)
        if (!isdigit(*i))
            *dest++ = *i;
}
```

Solution: Incorrect. `dest` already points to a block of memory, and the function should write the result there, instead of allocating memory itself.

- (d) */// @brief Removes the digits in the given string.
 /// @param str Points to a null-terminated byte string.
 /// @return int The number of digits removed.*

```
int remove_digits(char *str) {
    int digit_cnt = 0;
    for (char *i = str; *i != '\0'; ++i) {
        if (isdigit(*i)) {
            ++digit_cnt;
            for (char *j = i; *j != '\0'; ++j)
                *j = *(j + 1);
        }
    }
    return digit_cnt;
}
```

Solution: Incorrect. When two consecutive digits appear, the second will be skipped.

3. (15 points) (C++) Define a function `length_sum` that computes the sum of the length of all strings in a `std::vector<std::string>`. Fill in the blanks below.

```
#include <vector>
#include <string>
#include <cassert>
```

```
std::size_t length_sum(/* (a) */) {
    std::size_t sum = 0;
    for (/* (b) Use a range-based for loop. */)
        sum += /* (c) */;
    return sum;
}

int main() {
    std::vector<std::string> vs{"hello", "C++", "world"};
    assert(length_sum(vs) == 13); // This assertion should succeed.
}
```

(a) `const std::vector<std::string> &strings`

(b) `const std::string &s : strings`

Also correct: `const auto &s : strings`

Also correct: `auto &s : strings`

(c) `s.size()` or `s.length()` or `std::size(s)`

Also correct: `size(s)` (due to ADL)

CS100 Introduction to Programming
Spring 2024
Midterm Exam

Instructors: Lan Xu, Yuexin Ma

Time: May 27th 8:15-9:55

INSTRUCTIONS

Please read and follow the following instructions:

- You have 100 minutes to answer the questions.
- You are not allowed to bring any electronic devices including regular calculators.
- You are not allowed to discuss or share anything with others during the exam.
- You should write the answer to every problem in the dedicated box **clearly**.
- You should write **your name and your student ID** as indicated on the top of **each page** of the exam sheet.

Name	
Student ID	

1. (75 points) Fill in the blanks

The questions marked “[C]” are based on the C17 standard (ISO/IEC 9899:2018). The questions marked “[C++]” are based on the C++17 standard (ISO/IEC 14882:2017).

- (1) (10') The type of the string literal "hello" is char [6] in C, but const char [6] in C++.
- (2) (5') [C] Read two integers from input, separated by an unknown sequence of whitespaces: `scanf(____"%d%d"____, &a, &b).`

Solution: Any other equivalent way is also correct.

- (3) (5') [C] Given the following definition of a type representing a ghost in some game,

```
struct Ghost {
    int posX;
    int posY;
    char icon;
    bool isSmart;
};
```

allocate a block of memory to store `n` ghosts: `malloc(____n * sizeof(struct Ghost)____).`

Solution: Any other equivalent way is also correct.

- (4) (10') [C] Suppose we have the following declarations.

```
int array[10] = {0};
int *p1 = &array[2], *p2 = array + 6;
```

Is the type of the expression `p1 - p2` an integer type or a pointer type? - Integer (write “Integer” or “Pointer”).

The value of the expression `p1 - p2` is -4 (write “UB” if it is undefined behavior).

- (5) (15') [C++] Suppose `s` is an object of type `std::string`. Use `new` to create a `std::string` that is move-constructed from `s`:

```
auto t = new std::string(std::move(s));
```

The type of `t` is std::string *.

To destroy this string and deallocate the memory: delete t;

- (6) (15') The following code reads `n` strings from input, and stores them into a `std::vector<std::string>`. Fill in the blank with **the best way** of appending the string `s` to the end of `strings`.

```
std::vector<std::string> strings;
for (auto i = 0; i != n; ++i) {
    std::string s;
    std::cin >> s;
    strings.____push_back(std::move(s))____;
}
```

Suppose `k` is a variable of type `std::size_t`. Use a *range-based for loop* to traverse `strings` to count how many of the strings have length **less than** `k`. Fill in the blank with **the best answer**.

```
int cnt = 0;
for (____const auto &s : strings____) {
    if (s.size() < k)
```

```
    ++cnt;
}
```

`std::count_if` is a standard library function that accepts a **sequence** and a **unary predicate**, and returns the number of elements in the sequence for which the predicate returns **true**. Use `std::count_if` to rewrite the code above.

```
int cnt = std::count_if(
    _____ strings.begin(),
    _____ strings.end(),
    _____ [k](const std::string &s) { return s.size() < k; }
);
```

Solution: Any other equivalent way is also correct.

(7) Read the following code.

```
class Matrix {
    std::vector<std::vector<double>> mData;

public:
    explicit Matrix(std::vector<std::vector<double>> data) : mData{std::move(data)} {}

    double &operator()(std::size_t row, std::size_t col) {
        return mData[row][col];
    }

    double operator()(std::size_t row, std::size_t col) const {
        return mData[row][col];
    }

    /* (!!) */ friend Matrix operator-(const Matrix &) _____;

    Matrix operator+(const Matrix &rhs) _____ const _____;

    Matrix &operator+=(const Matrix &rhs) _____;
};
```

- i. (5') For each blank above, write a “**const**” where you think is necessary.
- ii. (5') Let **a** and **b** be two objects of type **Matrix**. Write an expression that uses the function marked “(!!)”: _____ **-a** _____.
- iii. (5') Let **a** be an object of type **Matrix**. How will you access the element on the **i**-th row and **j**-th column of **a** (the private members are not accessible)? - **a(i, j)** _____.

2. (20 points) Dynamic array

Consider the `Dynarray` class, which represents a dynamic array. It is the example given in the lectures, recitations and homework assignments.

```
class Dynarray {
public:
    Dynarray() = default;

    /* (a) */ explicit Dynarray(std::size_t n) : m_length{n}, m_storage{new int[n]{} } {}

    /* (b) */ Dynarray(std::size_t n, int x) : Dynarray(n) {
        std::fill_n(m_storage, n, x);
    }

    /* (c) */ Dynarray(const int *begin, const int *end) : Dynarray(end - begin) {
        std::copy(begin, end, m_storage);
    }

    /* (d) */ Dynarray(const Dynarray &other);

    Dynarray(Dynarray &&other) noexcept
        : m_length{other.m_length}, m_storage{other.m_storage} {
        other.m_length = 0;
        other.m_storage = nullptr;
    }

    ~Dynarray() { delete[] m_storage; }

    void swap(Dynarray &other) noexcept {
        std::swap(m_length, other.m_length);
        std::swap(m_storage, other.m_storage);
    }

    /* (e) */ Dynarray &operator=(Dynarray other) noexcept {
        swap(other);
        return *this;
    }

    // other members ...

private:
    std::size_t m_length{0};
    int *m_storage{nullptr};
};
```

- (1) (5') The way that the constructors (b) and (c) are defined is interesting: By writing `Dynarray(n)` and `Dynarray(end - begin)`, they *delegate* part of the initialization work to the constructor (a). Such constructors are called *delegating constructors*.

Can you define the constructor (d) as a *delegating constructor* too? Fill in the blank below.

```
Dynarray::Dynarray(const Dynarray &other)
    : Dynarray(____other.m_storage, other.m_storage + other.m_length____) {}
```

- (2) (5') What kind of assignment operator is (e)? Why?

Solution: It is both a copy assignment operator and a move assignment operator.

When assigning a `Dynarray` object with an expression *expr*, the parameter `other` will be initialized from *expr*, which performs copy-construction if *expr* is an lvalue and move-construction if it is an rvalue.

- (3) (10') Define a group of member functions `begin` and `end` for `Dynarray`, so that `Dynarray` can be traversed using a *range-based for loop*.

Write your answers by filling in the blanks below. If any of the functions is not needed, leave the blanks blank.

```
class Dynarray {
public:
    _____ begin() _____ { return m_storage; }
    _____ begin() _____ const { return m_storage; }
    _____ begin() _____
    _____ end() _____ { return m_storage + m_length; }
    _____ end() _____ const { return m_storage + m_length; }
    _____ end() _____

    // other members ...
};
```


3. (20 points) Understanding compiler's diagnostics

In each part of this question, you will be given a **code snippet** and **the compiler's diagnostics on the code**. The code is compiled using GCC 13 with `-Wall -Wpedantic -Wextra -std=c++17`. Explain **what the diagnostics mean** and **what is wrong with the code**.

Note: Some of you may be MSVC or Clang users and are unfamiliar with GCC's diagnostics, but don't worry. We are sure that the following diagnostics are easy to understand as long as you know the mistakes in the code.

(1) (10') Code:

```

3  class Action {
4      // ...
5  };
6
7  void runAction(std::unique_ptr<Action> action) {
8      // ...
9  }
10
11 void runTool() {
12     auto action = std::make_unique<Action>(/* ... */);
13     runAction(action);
14 }
```

Compiler's diagnostics:

```

tool.cpp: In function 'void runTool()':
tool.cpp:13:12: error: use of deleted function 'std::unique_ptr<Tp, _Dp>::unique_ptr(const
std::unique_ptr<Tp, _Dp&>) [with _Tp = Action; _Dp = std::default_delete<Action>]'
```

```

    13 |     runAction(action);
        |     ~~~~~^~~~~~
In file included from /usr/include/c++/13/memory:78,
        from tool.cpp:1:
/usr/include/c++/13/bits/unique_ptr.h:522:7: note: declared here
    522 |         unique_ptr(const unique_ptr&) = delete;
        |         ^~~~~~
tool.cpp:7:40: note:   initializing argument 1 of 'void runAction(std::unique_ptr<Action>)'
    7 | void runAction(std::unique_ptr<Action> action) {
        |                                     ~~~~~^~~~~~
```

The compiler says that the code uses a deleted function. What function is it? Why is it used by the code? How will you fix the error **if you can only modify the code in `runTool`**?

Solution: The deleted function this code uses is the copy constructor of `std::unique_ptr`. It is used because on line 13 `action` is an lvalue and is passed by value. To fix the error, we may pass `std::move(action)` as the argument, or we may just write `runAction(std::make_unique<Action>(/* ... */))` directly.

(2) (10') Code: Suppose `MatchFinder` is a default-constructible class type.

```
class ClassHandler {
    MatchFinder &mFinder;
    // ...

public:
    explicit ClassHandler(MatchFinder &finder) : mFinder{finder} {
        // ...
    }
};

class Tool {
    ClassHandler mClassHandler;
    MatchFinder mFinder;
    // ...

public:
    Tool() : mFinder{}, mClassHandler{mFinder} {
        // ...
    }
};
```

Compiler's diagnostics:

```
tool.cpp: In constructor 'Tool::Tool()':
tool.cpp:17:15: warning: 'Tool::mFinder' will be initialized after [-Wreorder]
   17 |     MatchFinder mFinder;
      |           ^~~~~~
tool.cpp:16:16: warning: 'ClassHandler Tool::mClassHandler' [-Wreorder]
   16 |     ClassHandler mClassHandler;
      |           ^~~~~~
tool.cpp:21:3: warning: when initialized here [-Wreorder]
   21 |     Tool() : mFinder{}, mClassHandler{mFinder} {
      |           ^~~~
```

What is the compiler warning against? Is there a real mistake in the code? How will you eliminate the warning and fix the mistake?

Solution: The warning says that the `mFinder` will be initialized after `mClassHandler`, but `mFinder` appears before `mClassHandler` in the constructor initializer list. When initializing `mClassHandler` using `mFinder`, `mFinder` is uninitialized, so the behavior is undefined. To eliminate the warning and fix the mistake, declare the member `mFinder` before `mClassHandler`.

4. (15 points) Inheritance

Suppose we have three different kinds of dialogue boxes.

```
class DialogueBox {
    std::string caption;
    // ...

public:
    virtual void display();
    virtual ~DialogueBox() = default;
    // ...
};

class Alert : public DialogueBox { /* ... */ } // An alert box

class Confirm : public DialogueBox { /* ... */ } // The "yes-or-no" box

class Progress : public DialogueBox { /* ... */ } // "loading ..."
```

All of the three kinds of dialogue boxes must support the `display()` interface.

- (1) (5') Suppose there is **no** well-defined “*default*” behavior for displaying a dialogue box. Select the (unique) best design for `DialogueBox::display`.

- A.

```
class DialogueBox {
public:
    virtual void display(); // without a definition
};
```
- B.

```
class DialogueBox {
public:
    virtual void display() = 0; // declared as pure virtual
};
```
- C.

```
class DialogueBox {
public:
    virtual void display() {} // defined and does nothing
};
```
- D.

```
class DialogueBox {
public:
    virtual void display() { // reports a runtime error
        error("Calling DialogueBox::display is not allowed.");
    }
};
```

- (2) (10') Suppose now there **is** a “*default*” behavior for displaying a dialogue box: Draw a box of default size, and then display the caption on it. It may be implemented as follows:

```
auto handle = Screen::drawBox(DefaultSize);
handle.setCaption(caption);
```

This default behavior should be provided by the base class `DialogueBox`, but

- we still want `DialogueBox` to be an abstract base class, and

- we don't want this default behavior to be inherited *automatically* and *silently*. That is, if a derived class wants to adopt this default behavior, it must write *something*.

How will you design `DialogBox` to achieve this goal? Write your answer in the code block below.

Note: For your convenience, you only need to write “`// DB`” to represent the code of the default behavior.

```
class DialogBox {
    std::string caption;
    // ...

public:
    virtual void display()

};
```

Suppose `Progress::display` wants to adopt the default behavior. Implement it in the code block below.

```
class Progress : public DialogBox {
public:
    void display() override {

    }
};
```

Solution: One possible way: a **protected** non-virtual function.

```
class DialogBox {
    std::string caption;
    // ...

public:
    virtual void display() = 0;

protected:
    void defaultDisplay() {
        // DB
    }
};
```

```
class Progress : public DialogBox {
public:
    void display() override {
        defaultDisplay();
    }
};
```

Another way: a pure `virtual` function with a definition.

```
class DialogueBox {  
    std::string caption;  
    // ...  
  
public:  
    virtual void display() = 0;  
};  
  
void DialogueBox::display() {  
    // DB  
}
```

```
class Progress : public DialogueBox {  
public:  
    void display() override {  
        DialogueBox::display();  
    }  
};
```